

A Retrieval-Augmented Multi-Agent System for Personalized Academic Scheduling

Martin Koome¹ and Mohamed Awud¹

¹Carnegie Mellon University, {mkoome, mawud}@andrew.cmu.edu

Group 14 | Agentic AI: Foundations and Applications | February 5, 2026

Abstract

Effective academic scheduling requires more than simple availability checking; it demands awareness of individual constraints such as energy patterns, deadlines, and task priorities. This paper presents the design and implementation of a Retrieval-Augmented Generation (RAG) enabled multi-agent system for personalized scheduling. The system integrates a domain-specific vector memory, a ReAct-style tool execution loop, and an independent verification module that evaluates the groundedness of generated responses. Our approach demonstrates how agentic reasoning, when combined with structured retrieval and verification, can produce reliable and context-aware scheduling decisions. Experimental evaluation shows robust navigation between retrieval and tool usage, high personalization accuracy, and effective hallucination control.

Introduction

This project implements a fully functional multi-agent academic scheduling system that operates over user-specific profile data. The core system is implemented in Python and consists of specialized agents responsible for planning, calendar management, and execution. The agents interact with both a Retrieval-Augmented Generation (RAG) system and the Google Calendar API to provide personalized scheduling recommendations grounded in real user data.

The repository includes the complete agent logic, tool implementations, and configuration required to connect to external data sources such as user profile embeddings stored in a vector database and live calendar data. The system is executable via a command-line interface and supports both interactive user queries and automated testing modes. This ensures that the agent behavior can be evaluated under realistic usage conditions while remaining reproducible for assessment.

Tooling Rationale

Custom tools were implemented to address the limitations of standard calendar systems for personalized academic planning. Calendar APIs alone do not capture user-specific constraints such as energy patterns, competencies, or strategic priorities.

To address this gap, a Strategic Memory Tool based on Retrieval-Augmented Generation (RAG) was developed. This tool leverages **ChromaDB** as a vector database to store and retrieve semantically relevant user profile data, including productivity patterns, deadlines, and task priorities. To ensure high-quality retrieval, the system employs a domain-specific, context-aware chunking strategy that preserves semantic relationships inherent in scheduling data. Rather than relying on generic text splitting, user information is partitioned along meaningful boundaries such as energy profiles, scheduling rules, competencies, and decision heuristics, enabling agents to retrieve complete and coherent context during reasoning.

Calendar Management Tools were extended with conflict detection, multi-calendar support, and timezone-safe scheduling to ensure feasibility and correctness. A Temporal Awareness Tool provides current time context for accurate deadline reasoning and time-sensitive decisions. Together, these tools allow the agents to dynamically navigate between retrieval and action in support of the project goals.

Trace Implementation

The system generates detailed implementation traces to demonstrate successful multi-step agent execution. These traces capture the agent’s reasoning flow, tool selection decisions, and final responses through structured logging.

Each trace explicitly documents intent classification, selection of the appropriate specialized agent, dynamic tool loading, retrieval and planning steps, and verification before producing a final response. The trace files provide clear visibility into decision points and module usage, satisfying the requirement for transparent agent reasoning and tool interaction.

Failure Analysis

One documented failure occurred when the Planner Agent selected only calendar tools and failed to consult the RAG system during certain planning queries. This resulted in generic scheduling recommendations that ignored user-specific energy constraints. The root cause was insufficient prompt constraints enforcing a mandatory tool usage order. The system prompt was updated to explicitly require retrieval from the RAG-based strategic memory prior to any calendar-based reasoning. In addition, execution-order logging was introduced to validate tool invocation sequences. These changes led to a substantial improvement in personalization quality and consistency.

A second failure was identified in time logic involving partial overlaps between events. In this case, the agent incorrectly marked a time interval as available when an existing event partially overlapped the requested period. This issue was uncovered through automated evaluation and resolved by formalizing overlap detection rules within the agent’s reasoning instructions.

A further implementation challenge arose during tool configuration and integration. Calendar Management Tools initially failed due to incorrect parameter usage and mismatches between expected input and output schema. These issues stemmed from incomplete alignment between the agent’s tool-calling logic and the formal tool specifications. Resolving this required a detailed review of the full tool documentation to understand required arguments, return structures, and edge cases. After aligning the agent prompts and tool wrappers with the documented schemas, tool invocation became reliable and reproducible. This experience highlighted the importance of strict schema adherence and precise interface design in agentic systems relying on dynamic tool execution.

Self-Evaluation and Verification Logic

A standalone evaluation system was developed to assess agent behavior independently of user interaction. The system executes a suite of automated test scenarios covering time logic, retrieval accuracy, calendar integration, and multi-tool coordination.

Agent responses are decomposed into individual factual claims, which are verified against evidence sources including calendar data, RAG retrievals, and temporal tools. A groundedness score is computed as the ratio of supported claims to total claims, enabling systematic detection of hallucinations.

Evaluation Results by Category

Table 1 summarizes system performance across all evaluation categories. The results highlight strong robustness in calendar tool integration and multi-tool coordination, with comparatively weaker performance in retrieval-only scenarios.

Table 1: Detailed Evaluation Performance by Category

Category	Scenarios	Passed	Failed	Avg. Score	Pass Rate
Time Logic & Conflict Detection	5	4	1	1.000	80%
RAG System Retrieval	5	3	2	0.700	60%
Calendar Tool Integration	5	5	0	1.000	100%
Multi-Tool Integration	3	3	0	1.000	100%

Across 18 evaluation scenarios, the system achieved an overall pass rate of 83.3% with an average groundedness score of 0.917, indicating strong factual grounding and reliable tool usage. Identified failures are explicitly documented and used to guide future improvements.

Time Logic and Conflict Detection achieved an 80% pass rate, with the single failure attributed to a partial overlap edge case. This failure was explicitly identified and documented, highlighting the effectiveness of the evaluation framework in surfacing non-trivial reasoning errors.

RAG System Retrieval exhibited a lower pass rate of 60%, primarily due to cases where the agent inferred answers from contextual cues or calendar data instead of explicitly querying the retrieval system. While responses remained largely grounded, this behavior indicates an opportunity to further enforce retrieval usage for queries involving deadlines, skills, and long-term constraints. Overall, the category-level analysis confirms strong implementation robustness while providing clear guidance for targeted system improvements.

Contribution Statement

This project was completed collaboratively with clearly defined responsibilities. Martin Koome led the design and implementation of the multi-agent architecture, developed the core agent logic and tool integration, implemented the Google Calendar interface, and conducted system testing, evaluation, and failure analysis.

Mohamed Awud focused on the design and implementation of the Retrieval-Augmented Generation system, including advanced semantic chunking strategies, metadata-driven retrieval, and optimization of personalized data access. We jointly contributed to system design decisions, evaluation methodology, performance analysis, and technical documentation.

Future Work

Building on the current multi-agent scheduling system, future work will focus on extending agent specialization to improve personalization and long-term planning. A **Review Agent** will analyze time-management patterns and generate periodic performance summaries to support informed scheduling decisions, while a **Recovery Agent** will propose adaptive plans to compensate for lost or unproductive time and improve overall efficiency.

Future iterations will also refine the system based on evaluation insights to enhance reliability and validation. We plan to fully containerize the system for simplified deployment and investigate the integration of open-source language models to improve privacy, transparency, and flexibility. These extensions aim to establish a more robust, adaptable, and user-centered academic planning platform.

The complete project codebase, including the current implementation and evaluation framework, is available at My Life in Blocks Repository, supporting ongoing development and the integration of future agent modules.